

Text Classification Pipeline Forensics: Diagnosing Which Changes to a Disaster-Tweet Classifier Are Trustworthy

Yufan XIAO

Thrust of Artificial Intelligence

Hong Kong University of Science and Technology (Guangzhou)

Guangzhou, China

yxiao986@connect.hkust-gz.edu.cn

Abstract—We build and forensically investigate a CPU-only TF-IDF and Logistic Regression classifier for the Kaggle Disaster Tweets task, treating the real question not as how high the held-out F1 is but as which pipeline changes are trustworthy. Five investigations—baseline reproduction, text normalization, a shallow-feature shortcut audit, decision-rule tuning, and a data-quality audit—are each run as a controlled experiment under a strict dev-decides and held-out-scored-once discipline, with every conclusion argued from the specific predictions that changed. The final pipeline (text-only with URL stripping and Logistic Regression at C equal to 3 and threshold 0.45) reaches a held-out F1 of 0.7653, above the reference 0.7574, reached by dev-selected steps rather than by tuning to the test set. The unifying finding is that a single score is not an explanation: a reference F1 is under-determined by many pipelines, a helpful preprocessing step removes a shortcut rather than noise, most of the task is solvable by artifacts redundant with the text, F1 and accuracy can move in opposite directions on one change, and label conflicts impose an error floor no model can cross.

Index Terms—text classification, TF-IDF, logistic regression, dataset artifacts, shortcut learning, precision-recall trade-off, data-quality audit, model evaluation

I. PROJECT PROBLEM AND GOAL

Text classifiers tend to look more interpretable than they are. A model’s F1 can improve because preprocessing removed noise, because a threshold happened to suit the metric, or because the model latched onto a shortcut in the data that will not survive contact with new examples—and the score alone cannot tell these apart [3]. When a number moves, the interesting question is not *how much* but *why*: which predictions changed, which errors were fixed, which new errors were introduced, and whether any of it matches the hypothesis under test.

The project builds a CPU-only classifier for the Kaggle Disaster Tweets task [1]—a binary problem where `target=1` marks a tweet describing a real disaster—against a reference contract that fixes the expected held-out F1 for a baseline pipeline. The scale is deliberately modest so that the difficulty lies not in engineering a strong model but in *earning the right to trust its results*.

The goal, and the report’s thesis, is a single claim demonstrated five times over: **a score is not an explanation**. Each

of the five investigations is a distinct way the held-out F1 can mislead a reader who takes it at face value—under-determination, shortcuts, redundancy, metric divergence, and a label-imposed ceiling—together with the prediction-level evidence needed to see past it.

II. METHODOLOGY

A. Data, split, and evaluation discipline

The data is the public Kaggle Disaster Tweets `train.csv` (7613 tweets; columns `id`, `keyword`, `location`, `text`, `target`). All work uses the provided split, keyed by `id` and stratified with seed 3102 into train 4567, dev 1523, and held-out 1523, with the positive rate of about 43% preserved. It is loaded by matching ids, never regenerated, and each script prints its per-split row and class counts against the documented figures so a loading error surfaces before any modeling conclusion.

The reported metric is F1 on the positive class (`target=1`), matching the reference contract. Two commitments govern its use. First, *decisions are separated from reporting*: preprocessing, thresholds, regularization, and model variants are chosen on dev, and held-out is scored exactly once per investigation, after that investigation’s decision is frozen; no configuration is selected because it did well on held-out. Where held-out must be consulted for a legitimate reason—reproducing the reference (Section III-A) or characterizing the data—it is labelled as diagnosis and not used to select the pipeline. Second, *conclusions are argued at the prediction level*: for every lever the analysis reports which dev examples flipped from correct to wrong and from wrong to correct, so a change in F1 is explained by a mechanism.

All work runs on CPU in a pinned conda environment (Python 3.11.15, scikit-learn 1.9.0 [2], pandas 3.0.3), recorded in `requirements.txt`; Section III-A shows the version pin is itself part of the experimental record. Every result is regenerable from the README commands, which write machine-checkable artifacts with stable schemas and ids.

B. Surface-feature profiling

Before choosing any preprocessing, the text was profiled for the surface features a normalization step might target—how often each occurs and how it correlates with the label (Table I). This makes the normalization choices principled rather than a generic checklist, and seeds the shortcut audit. URL and digit presence are the strongest positive correlates and are both posting-style artifacts; emoji never occur in the data; exclamation and mention presence correlate negatively; and tweet length rises monotonically with the positive rate.

TABLE I
SURFACE-FEATURE PROFILE (TRAIN.CSV, POSITIVE RATE 0.430)

Feature	Present	Pos. (pres.)	Pos. (abs.)	Δ
has_url	52.2%	0.547	0.302	+0.245
has_digit	59.4%	0.524	0.292	+0.232
has_hashtag	22.9%	0.492	0.411	+0.081
has_allcaps	25.3%	0.488	0.410	+0.078
has_mention	26.4%	0.332	0.465	−0.133
has_exclam.	9.4%	0.273	0.446	−0.173
has_emoji	0.0%	—	0.430	n/a

C. The five investigations and their methods

Each investigation follows the same controlled template—a stated **hypothesis**, a single **lever** changed against an otherwise frozen pipeline, a **dev experiment** analyzed through prediction changes, a single **held-out confirmation**, and an explicit **decision**—and each builds on the frozen decisions of the previous, so the pipeline accumulates one justified change at a time.

- 1) *Baseline reproduction.* Build a plain-default TF-IDF and Logistic Regression pipeline; probe one factor at a time (split, seed, convergence, version, vocabulary, regularization) to diagnose any gap from the reference metric.
- 2) *Text normalization.* Apply composable string transforms (URL, mention, hashtag, HTML, emoji, case) before vectorization, one at a time, measuring dev flips.
- 3) *Shortcut audit by deprivation.* Train models on one shallow feature at a time with no text tokens, so any F1 they reach comes entirely from that surface feature; classify each as legitimate, artifact, or mixed.
- 4) *Decision rule and model.* Sweep the threshold (the precision–recall curve), tune C on a logarithmic grid, test class weighting, and compare CPU classifiers suited to sparse TF-IDF (LinearSVC, MultinomialNB, SGD)—all selected on dev.
- 5) *Data-quality audit.* Detect exact-duplicate label conflicts and confident model–label disagreements, then adjudicate candidates by hand into four dispositions (fix, keep_but_flag, ambiguous, reject_false_positive), never modifying held-out labels.

III. MAIN EVIDENCE AND RESULTS

A. Baseline reproduction is under-determined

The plain-default pipeline reaches dev F1 0.7388 and held-out 0.7492, a 0.008 gap below the reference 0.7574 (tolerance ± 0.001). One-factor probes rule out the split (counts match exactly), the seed (the `lbfgs` solver is deterministic), convergence (≤ 19 iterations), and the library version—re-running under `scikit-learn` 1.3.2, 1.5.2, and 1.7.2 leaves held-out F1 within 0.0006. What moves the score toward the reference is TF-IDF vocabulary pruning (`min_df`) and regularization (C). A grid over standard settings then finds **five distinct configurations inside the ± 0.001 band, sharing no common story**, and none reproducing the reference to its full precision. The reference F1 is reproducible within tolerance by a conventional pipeline but does not identify one.

B. Normalization: only URL removal helps, and it removes a shortcut

Testing each transform on dev (Table II), only `strip_urls` clearly helps (dev 0.7388 to 0.7437). Hashtag-symbol and emoji removal produce *zero* flips—the default tokenizer already drops #, and the profile shows the data contains no emoji at all; case preservation *hurts* (vocabulary fragmentation). Frozen and scored once, `strip_urls` gives held-out F1 0.7492 to **0.7536** with flips `fixed_fp=29`, `fixed_fn=13`, `new_fp=7`, `new_fn=22`—a two-directional change analyzed in Section IV-A. A punctuation-preserving tokenizer was also tested (dev +0.0046) but declined: the gain comes from “!”, a negatively-correlated posting-style artifact.

TABLE II
NORMALIZATION TRANSFORMS (DEV), ONE AT A TIME

Variant	dev F1	Net flips
baseline (no cleaning)	0.7388	—
strip_urls	0.7437	+15
strip_mentions	0.7366	−3
strip_hashtag_symbol	0.7388	0
unescape_html	0.7384	~0
strip_emoji	0.7388	0
preserve_case	0.7311	−7

C. Shortcut ceiling and redundancy

Shallow-feature-only models (Table III) show a high shortcut ceiling: the `keyword` field alone reaches held-out F1 0.69—92% of the text model’s 0.749—a single “contains a link” bit reaches 0.61, and pure metadata reaches 0.55. Yet these are **redundant, not additive**: adding all shortcuts to the text model gives no dev-confirmed gain (dev drops 0.7388 to 0.7323 while held-out drifts up), because the text model already encodes them. No shortcut feature is adopted.

D. Decision rule: F1 up while accuracy falls

The threshold sweep (Table IV) is the precision–recall curve. The default 0.5 is precision-heavy (P=0.82, R=0.68, missing about 32% of disasters); dev F1 peaks at 0.45. On

TABLE III
SHALLOW-FEATURE-ONLY MODELS (HELD-OUT F1)

Model	F1
floor (majority class)	0.000
keyword_only	0.690
has_url_only	0.611
char_len_only	0.432
metadata_numeric_bundle	0.553
text_only (baseline)	0.749
text + all_shortcuts	0.756 (dev 0.732)

dev, $C=3$ is the best regularization (0.744 to 0.751), class weighting reaches 0.748, and no alternate classifier beats tuned LR (LinearSVC 0.735, MultinomialNB 0.729, SGD 0.753). The dev-winning combination $C=3$ and threshold 0.45, frozen and scored once, gives held-out **F1 0.7536 to 0.7653 while accuracy falls 0.8063 to 0.8011**—a divergence dissected in Section IV-C. Held-out 0.7653 now *exceeds* the reference 0.7574, reached by dev-selected steps.

TABLE IV
THRESHOLD SWEEP (DEV, LOGISTIC REGRESSION)

Threshold	Precision	Recall	F1
0.20	0.498	0.962	0.656
0.35	0.637	0.835	0.723
0.45	0.764	0.733	0.748
0.50 (default)	0.824	0.678	0.744
0.70	0.962	0.391	0.556

E. Data quality: an irreducible label ceiling

The audit (`data_quality_audit.csv`, 78 rows) finds 18 exact-duplicate groups with conflicting labels. Because identical text yields an identical prediction, any model must miss at least each group’s minority members—**22 rows are guaranteed wrong in the best case, 10 of them in held-out**. Confident model-label disagreements surface a further 16 fixable mislabels, all `target=1` tweets using a disaster keyword figuratively. Three disagreements were inspected and *kept* as `reject_false_positive`. Dispositions across the 78 rows: 47 fix, 18 ambiguous, 10 keep-but-flag (held-out), 3 reject.

IV. CASE ANALYSIS

A. The URL shortcut, in both directions

The 29 fixed false positives of `strip_urls` are non-disaster tweets whose only disaster-like feature was a link: a book title (“...Financial Meltdown by David Wiedemer `http` `http`...”, id 174), a save-the-wild-horses petition (id 386), a mixtape announcement (id 971). In training, real-disaster news links to articles, so the token `http` became a weak positive cue—a collection artifact, not disaster semantics. Crucially, the same shortcut propped up genuine disasters: the roughly 22 new false negatives are real events that only crossed threshold *because* of URL weight (“...Catastrophic Eruption - The New Yorker `http`...”, id 2193; “Roof collapsed a bowling alley...`http`...”, id 2364). Removing a shortcut therefore flips

predictions **both ways**; a pure noise removal would only fix errors. This two-directional signature is how one distinguishes the two.

B. The keyword weight split: legitimate vs. memorized

The keyword-only model’s weights split cleanly. The strong *positive* keywords are concrete, literal disaster nouns—*derailment, debris, wreckage, typhoon, bombing, evacuated, terrorist*. The strong *negative* keywords are disaster-adjacent words used figuratively in this dataset—*aftershock, bloody, blazing, hellfire, explode, blew up*. The positive side is legitimate task signal; the negative side is the model memorizing that tweets tagged `blazing` or `explode` *happened to be* mostly non-disaster here—memorization of Kaggle’s per-keyword selection, not language understanding, which would not transfer. This is why `keyword` is classified as *mixed* rather than adopted, and it predicts exactly the mislabels found in Section IV-D.

C. The precision–recall trade at the row level

The F1-up and accuracy-down result is legible only in the confusion matrix (Table V). Lowering the threshold from 0.5 to 0.45 recovers **43 missed disasters** (FN to TP) at the cost of **51 more false alarms** (TN to FP). Accuracy counts all 1523 rows equally, so 43 gains against 51 losses is a net -8 correct and it falls. The F1 formula $2TP/(2TP + FP + FN)$ has no TN term, so it sees only the positive class, where recall climbs (0.690 to 0.755) far more than precision dips (0.831 to 0.776), and it rises. The metrics diverge because they encode different values—accuracy treats a missed disaster and a false alarm as equally bad; F1 treats catching disasters as the goal. For a task whose negatives are an easy 57% majority (an all-negative model scores 0.57 accuracy at 0.0 F1), F1 is the appropriate metric, which is why the contract uses it. Reporting either number alone would mislead; the flip breakdown makes the trade legible.

TABLE V
HELD-OUT CONFUSION MATRICES (654 DISASTERS, 869 NON-DISASTERS)

Pipeline	TP	FP	FN	TN	F1	Acc
<code>strip_urls</code> , thr 0.5	451	92	203	777	0.754	0.806
$C=3$, thr 0.45	494	143	160	726	0.765	0.801

D. Mislabels and the audit’s brake

Representative fixes and rejects show the judgment the audit requires. *Fixes*: “My back is so sunburned” tagged `hurricane` and labeled 1, corrected to 0; a Star Wars toy listing tagged `battle`; a fish-tank ornament tagged `sunk`. These are the concrete label noise behind the negative-weight keywords of Section IV-B. *Rejects (kept, not flipped)*: “I’m On Fire.” labeled 0 with model probability 0.92—an idiom, the 0 is correct and the model over-triggers on `fire`; “Police expand search for missing pregnant woman” labeled 0—a single missing-person search is defensibly not a disaster; #Megaquake commentary labeled 1 that the model misreads.

Reporting these three as `reject_false_positive` rather than flipping them is what separates an audit from a list of every disagreement—and it shows the dev metric is measured against noisy labels, so the true operating point is slightly better than reported.

V. DIFFICULTIES AND SOLUTIONS

1) The reference baseline could not be uniquely reproduced. The contract gives a held-out F1 to sixteen digits, but the plain-default pipeline lands 0.008 below it and no configuration matches exactly. The difficulty was knowing when to stop chasing the number. It was resolved by reframing the task from “hit 0.7574” to “explain the discrepancy”: one-factor probes ruled out split, seed, convergence, and version, and a grid showed five standard configurations reach the ± 0.001 band without sharing a story. The honest conclusion—the metric under-determines the pipeline—is stronger than a forced match, and is verifiable in `ticket1_probe.csv` and `ticket1_versions.csv`.

2) A silent baseline pollution was caught by a harness check. The first `baseline.py` produced by the coding assistant had quietly baked `ngram_range=(1, 2)` and `min_df=2` into the “default” pipeline, which would have stolen the very levers the later investigations exist to test. It was detected by requiring the no-op/default configuration to reproduce a frozen dev F1 (0.7388) exactly; the mismatch surfaced the polluted parameters, and the baseline was rebuilt plain and re-verified. The lesson—a baseline is trustworthy only once a known-null reproduces a known number—became a standing check.

3) An MKL illegal-instruction crash blocked the version experiment. Testing whether library version explained the baseline gap required isolated environments with older scikit-learn, whose conda-forge MKL build hard-crashes (illegal instruction `0xC06D007F`) on any BLAS call on this machine—an i9-14900HX whose fused-off AVX-512 the MKL CPU dispatch mishandles. The failure fired even on a bare NumPy matmul; it was diagnosed to the BLAS backend and fixed by forcing `libblas==*openblas` in each environment. Ruling version out as a cause thus depended on first solving an environment bug.

VI. AI USAGE DECLARATION

An AI coding assistant was used substantially; the interaction log is in `logs/chat.md`. Its role was implementation and enumeration under the author’s direction: writing and running the pipeline and experiment scripts, producing the machine-checkable artifacts, and surfacing candidate rows for the data-quality audit. The experimental design—which hypothesis each investigation tests, which lever to change, what counts as evidence—and the interpretation of every result were directed and owned by the author. Outputs were verified rather than trusted:

- *Harness checks.* Each script confirms split counts against the documented figures, and the default configuration was

required to reproduce a frozen dev F1—the check that caught the polluted baseline (Difficulty 2).

- *Discipline enforced by the author.* The dev-decides and held-out-once rule was maintained across all investigations, including deliberately not computing held-out scores for rejected candidates in the decision-rule study. Two clarifying questions the assistant raised about held-out handling were resolved toward the stricter option.
- *Judgments made by hand.* Every audit disposition—fix, ambiguous, `reject_false_positive`—was decided by reading the actual tweet, not delegated to the tool; the assistant produced only the candidate list, and no judgment was fabricated.
- *Key numbers re-derived.* Central results (the confusion-matrix reconstruction behind the F1-up and accuracy-down finding, and the guaranteed-error floor) were checked independently of the assistant’s reported summaries.

VII. DISCUSSION AND LIMITATIONS

The five investigations, read together, are one argument—a **score is not an explanation**—demonstrated on five different ways the held-out F1 can mislead (Table VI). The arc has a direction: it begins at the *input* to the score (a reference F1 that does not pin a pipeline) and ends at the *labels* the score is computed against (an error floor no model can cross), passing through the model’s features (a shortcut removed, quantified, and classified) and its decision rule (two metrics pointing opposite ways). At every step the summary number was insufficient and the same instrument resolved it: prediction-level evidence under a dev-decides and held-out-once discipline.

TABLE VI
ONE THESIS, FIVE FACETS

Investigation	What the number hid
Baseline	the reference is under-determined
Normalization	<code>strip_urls</code> removes a shortcut, not noise
Shortcut audit	surface artifacts, redundant with text
Decision rule	accuracy <i>fell</i> on the same move
Data quality	labels cap the score (22 rows)

Several limitations bound the conclusions. The shortcut and artifact findings are specific to how this dataset was collected and illustrate a general risk without quantifying it elsewhere. The legitimate-vs-artifact call for `keyword` and the audit dispositions are reasoned judgments on short, figurative text, not measurements. Although each investigation freezes before touching held-out, five of them touch it in total, so the discipline bounds per-investigation peeking rather than the project-level fact that the same 1523 rows informed several reported numbers. The scope is CPU linear models on TF-IDF only; the reference pipeline’s exact configuration remains unidentified, and the adopted threshold is calibrated to this data’s class balance—it generalized within the dataset (the dev-optimal 0.45 also improved held-out) but would need recalibration on a differently balanced stream, affecting only the operating point, not the model’s ranking.

VIII. CONCLUSION

The final pipeline—text-only, URL stripping, Logistic Regression at $C=3$ and threshold 0.45—reaches a held-out F1 of 0.7653, above the reference, with every step chosen on dev, confirmed once on held-out, and explained through the specific predictions that changed. More important than the number is what stands behind it. A plain baseline that “reproduces” a reference need not share its pipeline; a preprocessing step that raises F1 may be removing a shortcut and reshaping predictions in both directions; most of a task can be solvable by artifacts that add nothing to a real model; two summary metrics can disagree about whether the same change helped; and the labels themselves can cap the score below 1.0. The pipeline also declines every artifact-based gain on offer—metadata features, a punctuation cue, class weighting—on the principle that a robust, explained 0.765 is worth more than an inflated, unexplained one. The lesson generalizes beyond this dataset: trust in a result comes not from the score but from the prediction-level evidence and the discipline behind it.

REFERENCES

- [1] Kaggle, “Natural Language Processing with Disaster Tweets,” 2019. [Online]. Available: <https://www.kaggle.com/competitions/nlp-getting-started/data>. Labeled `train.csv` mirror: <https://raw.githubusercontent.com/ucbrise/kaggle-nlp-disasters/master/data/train.csv>
- [2] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [3] R. Geirhos et al., “Shortcut learning in deep neural networks,” *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.